Transactions and ACID

Chapter 05 - Sub-Chapter 04

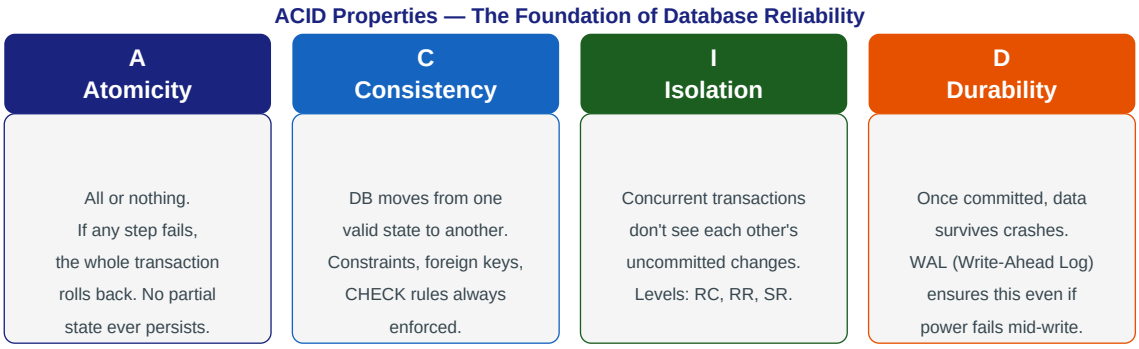
SQL Databases Series

ACID Properties	Atomicity, Consistency, Isolation, Durability
Transaction Control	BEGIN, COMMIT, ROLLBACK, SAVEPOINT, @Transactional
Propagation	REQUIRED, REQUIRES_NEW, self-invocation trap
Locking	SELECT FOR UPDATE, @Version, optimistic vs pessimistic

1 ACID Properties

Why databases are trustworthy

Imagine transferring $200 from your savings to your checking account. Two operations: debit savings, credit checking. What if the server crashes between them? ACID properties ensure this never leaves your money in limbo.

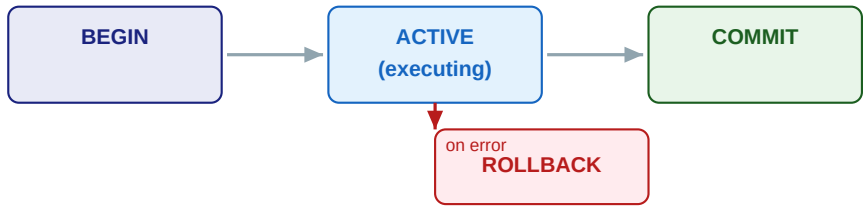


The classic bank transfer — all ACID properties in action:

```
-- Bank transfer: move $200 from account 1 to account 2
BEGIN;  -- start transaction

    -- Step 1: debit source account
    UPDATE accounts SET balance = balance - 200 WHERE account_id = 1;

    -- Step 2: credit destination account
    UPDATE accounts SET balance = balance + 200 WHERE account_id = 2;

    -- Step 3: log the transfer
    INSERT INTO transfers (from_id, to_id, amount, ts)
    VALUES (1, 2, 200, NOW());

COMMIT;  -- all three steps succeed together

-- If ANY step fails (e.g., constraint violation: balance < 0):
ROLLBACK;  -- ALL changes are undone -- as if nothing happened

-- ACID guarantees:
-- A: Either all 3 statements commit or none do
-- C: Balance constraint (CHECK balance >= 0) enforced atomically
-- I: Another session sees either old balances or new -- never intermediate
-- D: After COMMIT, data survives power failure (WAL written to disk)
```

2 Transaction Control
BEGIN, COMMIT, ROLLBACK, SAVEPOINT

Transaction Lifecycle — BEGIN to COMMIT or ROLLBACK



COMMIT: writes WAL, makes changes visible | ROLLBACK: discards all changes since BEGIN

Savepoints — partial rollback within a transaction:

A SAVEPOINT marks a point within a transaction you can roll back to without aborting the entire transaction. Useful for try-retry logic inside a large batch.

```
BEGIN;

    INSERT INTO orders (user_id, total) VALUES (42, 89.99);
    SAVEPOINT after_order;   -- mark this point

    INSERT INTO order_items (order_id, sku, qty) VALUES (currval('orders_order_id_seq'), 'X', 0);
    -- ERROR: CHECK constraint violation (qty must be > 0)

    ROLLBACK TO SAVEPOINT after_order;  -- undo just the bad insert
    -- Order row is still there!

    INSERT INTO order_items (order_id, sku, qty) VALUES (currval('orders_order_id_seq'), 'X', 1);
    -- This one is fine

COMMIT;  -- order + valid item committed, bad item never happened
```

Spring @Transactional — declarative transaction management:

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Isolation;

@Service
public class OrderService {

    private final OrderRepository orderRepo;
    private final InventoryService inventoryService;
    private final PaymentService paymentService;

    // @Transactional wraps the method in a BEGIN...COMMIT block
    @Transactional
    public Order placeOrder(Long userId, List<OrderItemRequest> items) {
        // Step 1: create order
        Order order = orderRepo.save(new Order(userId, "PENDING"));

        // Step 2: reserve inventory (runs in SAME transaction)
        inventoryService.reserve(order.getId(), items);

        // Step 3: process payment (runs in SAME transaction)
        paymentService.charge(userId, order.getTotal());
```

```
        // If ANY step throws RuntimeException: ROLLBACK
        // All three operations are atomic!
        order.setStatus("CONFIRMED");
        return orderRepo.save(order);
    }

    // readOnly=true: hint to Hibernate not to flush, uses read replica if configured
    @Transactional(readOnly = true)
    public List<Order> getUserOrders(Long userId) {
        return orderRepo.findByUserId(userId);
    }

    // Custom timeout and isolation
    @Transactional(timeout = 30,
                   isolation = Isolation.READ_COMMITTED,
                   rollbackFor = PaymentException.class)
    public void processRefund(Long orderId) { ... }
}
```

3 Transaction Propagation

How nested @Transactional methods behave

When one @Transactional method calls another @Transactional method, Spring must decide: does the inner method join the outer transaction, start a new one, or something else? This is controlled by the **propagation** attribute.

```
import org.springframework.transaction.annotation.Propagation;

// REQUIRED (default): join existing transaction, or start one if none exists
// Most methods should use this
@Transactional(propagation = Propagation.REQUIRED)
public void doWork() { ... }

// REQUIRES_NEW: always start a NEW transaction, suspend the outer one
// Use for: audit logging, notification sending -- should commit independently
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void saveAuditLog(AuditEvent event) {
    auditRepo.save(event);
    // commits even if the outer transaction rolls back!
}

// SUPPORTS: join existing transaction if present, otherwise run non-transactional
@Transactional(propagation = Propagation.SUPPORTS)
public List<Order> readOrders(Long userId) { ... }  // read-only, flexible

// MANDATORY: must be called within an existing transaction (throws if none)
@Transactional(propagation = Propagation.MANDATORY)
public void deductInventory(Long productId, int qty) {
    // Enforces that caller has opened a transaction
}

// NOT_SUPPORTED: suspend any existing transaction, run without one
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void sendEmail(String to, String body) {
    // Email sending shouldn't hold DB locks
}

// NEVER: throw exception if called within a transaction
@Transactional(propagation = Propagation.NEVER)
public void longRunningReport() { ... }
```

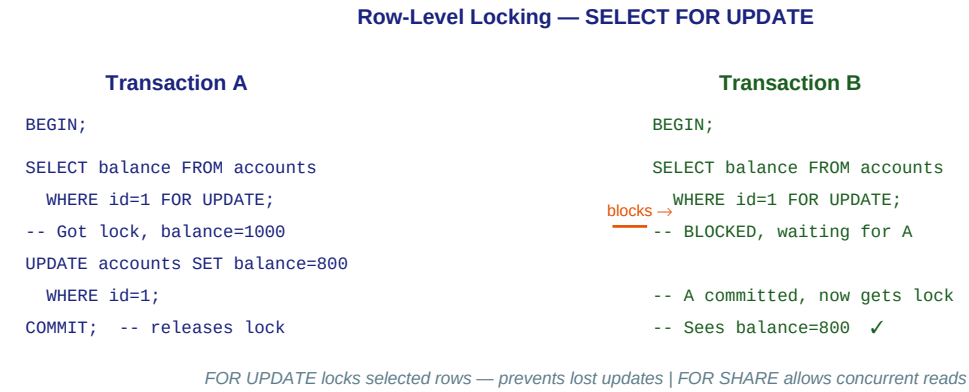
Common mistake: self-invocation bypasses @Transactional:

```
@Service
public class OrderService {

    @Transactional
    public void placeOrder(OrderRequest req) {
        // ...
        notifyUser(req.getUserId());   // PROBLEM: self-call bypasses proxy!
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void notifyUser(Long userId) {
        // This annotation is IGNORED when called from the same bean!
        // Spring AOP proxy is bypassed on self-invocation
    }
}
```

```
// Fix 1: inject the service into itself (Spring handles circular injection)
@Service
public class OrderService {
    @Autowired private OrderService self;  // inject proxy

    public void placeOrder(OrderRequest req) {
        self.notifyUser(req.getUserId());  // goes through proxy, works!
    }
}

// Fix 2: move notifyUser to a separate @Service class
```

4 Locking and Concurrency

Optimistic vs Pessimistic locking



Pessimistic locking — lock at DB level (SELECT FOR UPDATE):

```
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Query;
import jakarta.persistence.LockModeType;
import java.util.Optional;

@Repository
public interface AccountRepository extends JpaRepository<Account, Long> {

    // Translates to: SELECT ... FROM accounts WHERE id=? FOR UPDATE
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT a FROM Account a WHERE a.id = :id")
    Optional<Account> findByIdForUpdate(Long id);
}

@Service
public class TransferService {

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        // Lock both accounts in consistent order (prevent deadlock!)
        Long first  = Math.min(fromId, toId);
        Long second = Math.max(fromId, toId);
        Account a1 = accountRepo.findByIdForUpdate(first).orElseThrow();
        Account a2 = accountRepo.findByIdForUpdate(second).orElseThrow();

        Account from = (a1.getId().equals(fromId)) ? a1 : a2;
        Account to   = (a1.getId().equals(toId))   ? a1 : a2;

        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException("Balance too low");
        }
        from.setBalance(from.getBalance().subtract(amount));
        to.setBalance(to.getBalance().add(amount));
        // Locks released on COMMIT
    }
}
```

Optimistic locking — detect conflicts at commit time (no DB lock):

```
import jakarta.persistence.Version;
import jakarta.persistence.Entity;
```

```
@Entity
public class Product {
    @Id private Long id;
    private int stockQty;

    @Version                    // Hibernate manages this column
    private Long version;       // incremented on every update
}

// When two transactions try to update concurrently:
// T1: reads version=5, updates version to 6
// T2: reads version=5 (same!), tries to update -- version is now 6!
//     Hibernate throws OptimisticLockException -- no lost update!

@Service
public class InventoryService {

    @Transactional
    @Retryable(value = ObjectOptimisticLockingFailureException.class, maxAttempts = 3)
    public void reserveStock(Long productId, int qty) {
        Product product = productRepo.findById(productId).orElseThrow();
        if (product.getStockQty() < qty) throw new InsufficientStockException();
        product.setStockQty(product.getStockQty() - qty);
        productRepo.save(product);  // throws if version changed since read
    }
}
// Optimistic: great for low-contention (most updates don't conflict)
// Pessimistic: great for high-contention (many concurrent updates to same row)
```

Key Rules

- @Transactional is method-level -- every public method that modifies data should have it.
- Use readOnly=true for SELECT-only transactions -- allows read-replica routing and skips flush.
- REQUIRES_NEW for audit logs and notifications -- they should commit independently.
- Always lock rows in a consistent order (by ID) to prevent deadlocks.
- Use optimistic locking (@Version) for low-contention -- pessimistic for high-contention.
- Never use @Transactional on private methods or call @Transactional methods on self (use proxy).